DATA STRUCTURES AND APPLICATIONS

MODULE 1-2(Till Stacks) – COMPLETE THEORY NOTES

Course Code: 1BCS305 | III Semester – CSE (AI) | Duration: 9 Hours

Syllabus coverage:

MODULE – I Introduction to Data Structures: Data Structures, Classifications of Data Structures, Data structure operations: Traversing,
inserting, deleting, searching and sorting.
Structures: Definition, Types, Array of structures and programming examples.
Stacks: Stack, Stack Operations, Array Representation of Stacks, Different types of expression: Infix, Postfix and Prefix.
Case Study: Real world Applications

MODULE – II Stack applications (infix-to-postfix, infix-to-prefix, postfix evaluation, recursion);

Learning outcomes

- Define data structures, classify them, and perform traversing, inserting, deleting, searching and sorting on arrays.
- Declare structures and arrays of structures in C and access their members.
- Implement a stack using an array with push, pop and peek, including overflow and underflow checks.
- Identify infix, postfix and prefix expressions and convert simple ones by hand.
- Convert infix expressions to postfix and prefix using a stack.
- Evaluate postfix/suffix expressions using a stack.
- Explain recursion and trace recursive calls.
- Implement array queues and explain enqueue/dequeue.
- Explain circular queues, deques and priority queues.
- Choose a suitable structure for a real-world problem.

MODULE I – INTRODUCTION TO DATA STRUCTURES, STRUCTURES AND STACKS

1. Introduction to data structures

1.1 Basic terms

A **data structure** is a systematic way of organizing, storing and accessing data so that operations such as searching, inserting and deleting can be done efficiently. Choosing the right data structure decides how fast and how memory-friendly a program is.

Term	Meaning	Example
Data	Raw facts or values	85, "Asha", 3.14
Data item	Smallest unit of data that has a meaning	Roll number of a student
Record	Collection of related data items	Roll no + name + marks of one student
Data type	Set of values together with the operations allowed on them	int, float, char
Data structure	Organized collection of data together with the operations on it	Array, stack, structure
Abstract Data Type (ADT)	Model that defines what operations are done, not how they are implemented	Stack ADT: push, pop, peek

1.2 Classification of data structures

Basis	Category	Description	Examples
Nature of data	Primitive	Basic types built into the language; hold a single value	int, float, char, double, pointer
	Non-primitive	Built from primitive types; hold a collection of values	Array, structure, stack, queue, linked list, tree, graph
Arrangement	Linear	Elements form a sequence; each element has one predecessor and one successor	Array, stack, queue, linked list
	Non-linear	Elements are arranged hierarchically or as a network	Tree (file system), graph (road map)
Memory size	Static	Size fixed at compile time	Array
	Dynamic	Size can grow or shrink at run time	Linked list, dynamically allocated arrays
Type of elements	Homogeneous	All elements have the same data type	Array
	Heterogeneous	Elements may have different data types	Structure

Trees and graphs are studied in later modules; in this module the focus is on arrays, structures and stacks.

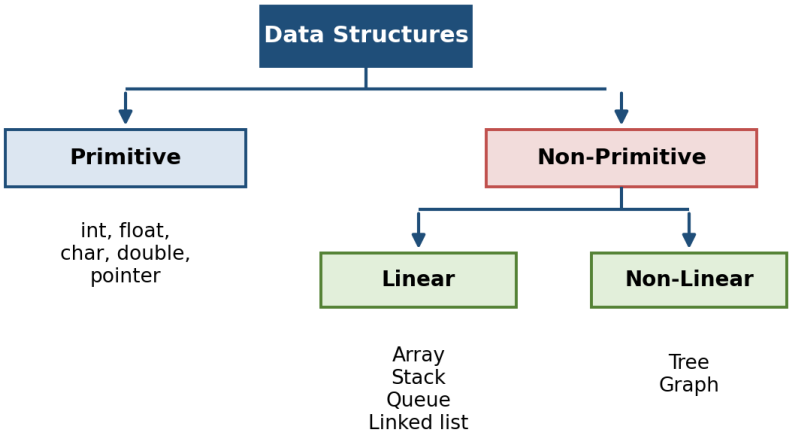


Fig. 1: Classification of data structures

1.3 Data structure operations

Operation	Meaning	Example on array 10, 20, 30, 40
Traversing	Visiting every element exactly once to process it	Print all four elements
Inserting	Adding a new element at a given position	Insert 25 at index 2: 10, 20, 25, 30, 40
Deleting	Removing an element and closing the gap	Delete 20: 10, 30, 40
Searching	Finding the location of a given value	Search 30: found at index 2
Sorting	Arranging elements in ascending or descending order	40, 30, 20, 10 (descending)

1.4 Insertion and deletion in an array

Array elements occupy consecutive memory locations, so inserting or deleting in the middle requires shifting the elements that follow.

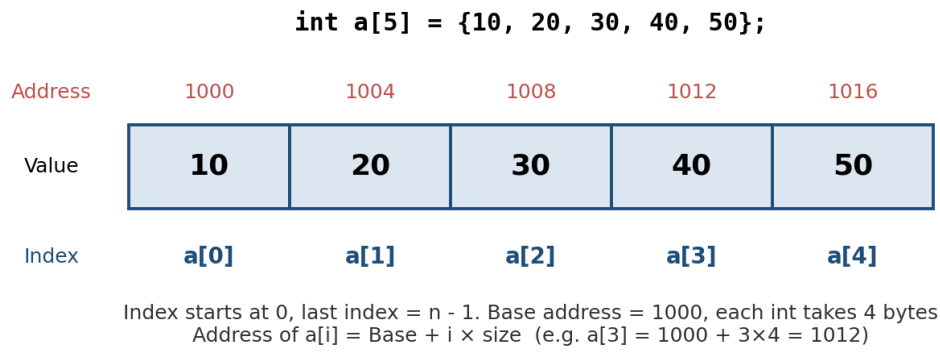
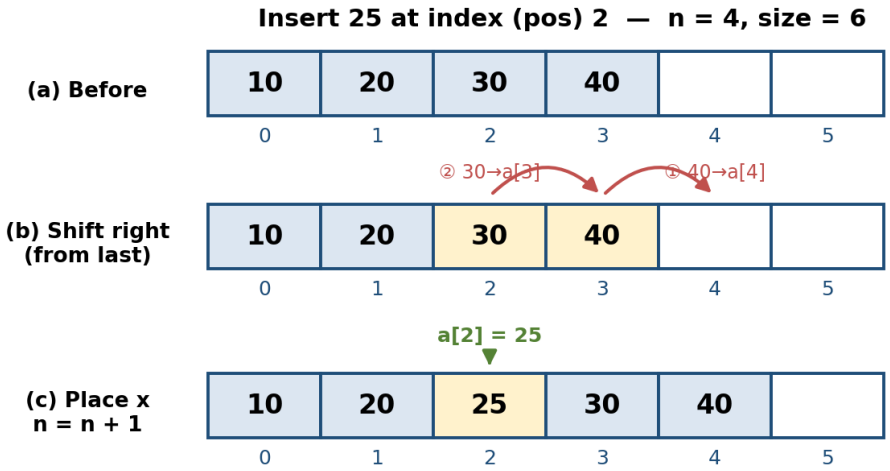


Fig. 2: Array in memory: index, value and address

Insert x at position pos (0-based) in an array of n elements:

1. Check that the array has room (n < size) and that 0 <= pos <= n.
2. Shift elements right: for i = n-1 down to pos, set a[i+1] = a[i].
3. Store the new value: a[pos] = x.
4. Increase the count: n = n + 1.

```
for (i = n - 1; i >= pos; i--)
    a[i + 1] = a[i];
a[pos] = x;
n++;
```



Result: 10, 20, 25, 30, 40 | Elements from index pos to n-1 move one place right → O(n)

Fig. 3: Insertion into an array (shift right, then place)

Delete the element at position pos:

1. Check that 0 <= pos < n.
2. Shift elements left: for i = pos to n-2, set a[i] = a[i+1].

3. Decrease the count: n = n - 1.

```
for (i = pos; i < n - 1; i++)
    a[i] = a[i + 1];
n--;
```

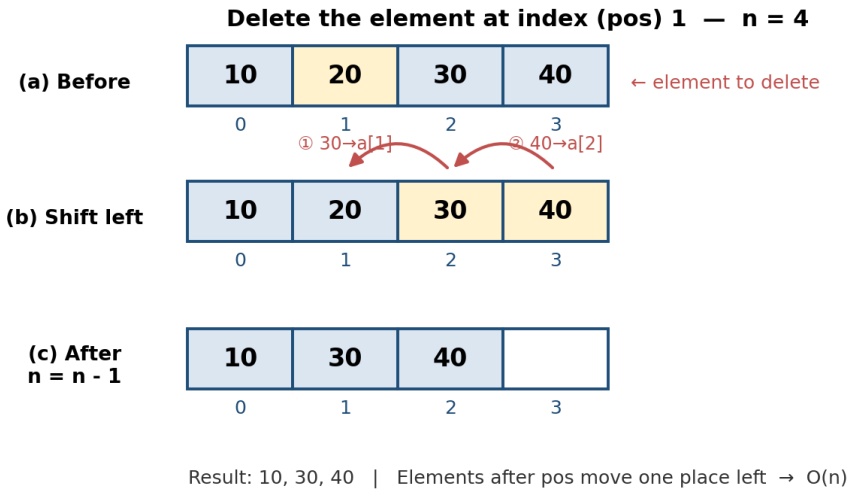


Fig. 4: Deletion from an array (shift left)

Example: inserting 25 at index 2 of 10, 20, 30, 40 first moves 40 and 30 one place right, then writes 25 at index 2, giving 10, 20, 25, 30, 40.

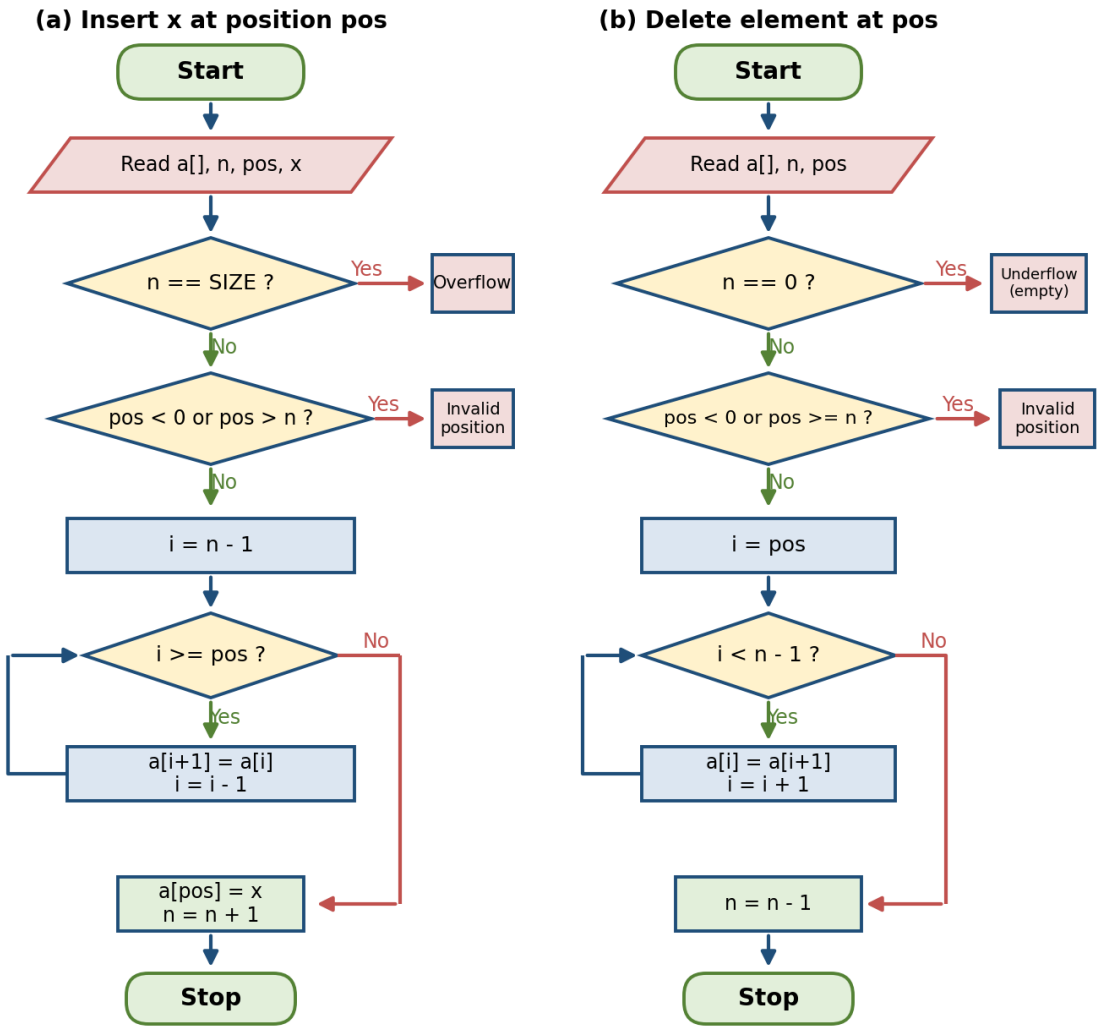


Fig. 5: Flowcharts for array insertion and deletion

1.5 Time complexity of array operations

Operation	Best / typical case	Worst case	Reason
Traversing	O(n)	O(n)	Every element is visited once
Insert at end	O(1)	O(1)	No shifting needed
Insert at position / at beginning	O(1) at the end	O(n)	Elements after pos must be shifted
Delete at position	O(1) for the last element	O(n)	Elements after pos must be shifted
Linear search	O(1)	O(n)	Target may be last or absent
Binary search (sorted array)	O(1)	O(log n)	Search range halves each step

2. Structures in C

2.1 Definition and declaration

An array stores elements of the same type. A **structure** groups variables of different types under one name, so that all details of one entity (for example a student) can be handled together. Each variable inside a structure is called a **member**.

```
struct student
{
    int   roll;
    char  name[20];
    float marks;
};            /* semicolon is mandatory */

struct student s1, s2;   /* declaring structure variables */
```

The structure definition only creates a new data type; memory is allocated when variables are declared. The size of a structure is at least the sum of the sizes of its members (the compiler may add padding for alignment).

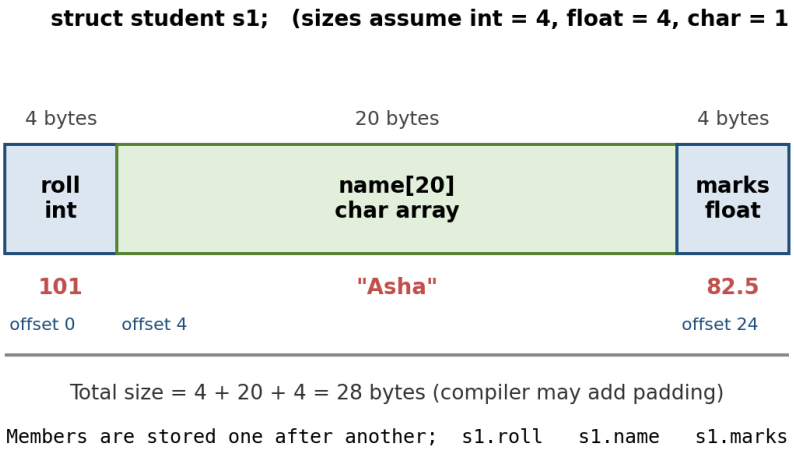


Fig. 6: Memory layout of a structure variable

2.2 Accessing members

Use the **dot operator (.)** with a structure variable and the **arrow operator (->)** with a pointer to a structure.

```
struct student s1 = {101, "Asha", 82.5};
struct student *p = &s1;

printf("%d %s %.1f\n", s1.roll, s1.name, s1.marks);   /* dot operator   */
printf("%d %s %.1f\n", p->roll, p->name, p->marks);   /* arrow operator */
/* p->marks is the same as (*p).marks */
```

2.3 typedef with structures

typedef gives a structure a shorter type name so that the keyword struct need not be repeated.

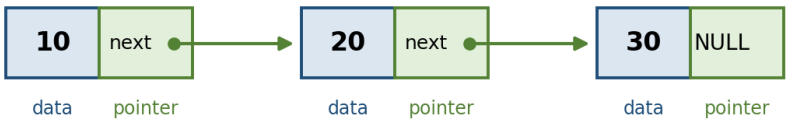
```
typedef struct
{
    int   roll;
    char  name[20];
    float marks;
} STUDENT;

STUDENT s1, s2;
```

2.4 Types of structures

Type	Description	Example
Simple structure	Members are basic data types or arrays	struct student { int roll; float marks; };
Nested structure	A member is itself a structure	e.dob.year, where dob is a struct date
Array of structures	Many records of the same structure type	struct student s[50];
Pointer to structure	Pointer holds the address of a structure; members accessed with ->	struct student *p = &s1;
Self-referential structure	Contains a pointer to a structure of its own type; basis of linked lists	struct node { int data; struct node *next; };

```
struct node { int data; struct node *next; };
```



A structure that contains a pointer to its own type: the basis of linked lists

Fig. 7: Self-referential structure (node of a linked list)

Nested structure example:

```
struct date
{
    int day, month, year;
};

struct employee
{
    int  id;
    char name[20];
    struct date dob;      /* nested structure */
};

struct employee e = {1, "Ravi", {15, 8, 2001}};
printf("%d", e.dob.year);   /* prints 2001 */
```

2.5 Array of structures

To store details of many students we declare an array whose every element is a structure. Member access is s[i].member. The program below reads the details of N students, prints them, and finds the average marks and the topper.

struct student s[3];

s[0]	
roll	1
name	Asha
marks	82.5

s[1]	
roll	2
name	Ravi
marks	74.0

s[2]	
roll	3
name	Meena
marks	91.0

`s[1].name → "Ravi"` `s[2].marks → 91.0` `s[0].roll → 1`

Elements s[0], s[1], s[2] occupy consecutive memory, each one a complete structure

Fig. 8: Array of structures

```
#include <stdio.h>
#define N 3

struct student
{
    int   roll;
    char  name[20];
    float marks;
};

int main()
{
    struct student s[N];
    int   i, top = 0;
    float total = 0;

    for (i = 0; i < N; i++)
    {
        printf("Enter roll, name, marks: ");
        scanf("%d %19s %f", &s[i].roll, s[i].name, &s[i].marks);
        total += s[i].marks;
        if (s[i].marks > s[top].marks)
            top = i;
    }

    printf("\nRoll\tName\tMarks\n");
    for (i = 0; i < N; i++)
        printf("%d\t%s\t%.2f\n", s[i].roll, s[i].name, s[i].marks);

    printf("Average = %.2f\n", total / N);
    printf("Topper  = %s (%.2f)\n", s[top].name, s[top].marks);
    return 0;
}
```

Sample run with input 1 Asha 82.5, 2 Ravi 74, 3 Meena 91:

```
Roll    Name    Marks
1       Asha    82.50
```

```
2       Ravi    74.00
3       Meena   91.00
Average = 82.50
Topper  = Meena (91.00)
```

Here traversing is the two loops, and searching (topper) is done while reading. Inserting, deleting and sorting records follow the array techniques of Section 1.4, applied to whole structure elements.

2.6 Structure versus union

Point	Structure	Union
Keyword	struct	union
Memory	Separate memory for every member (sum of sizes)	One shared block, size of the largest member
Access	All members hold values at the same time	Only one member holds a valid value at a time
Typical use	Records such as student or employee	Saving memory when only one of several forms of data is needed

2.7 Common mistakes

- Forgetting the semicolon after the closing brace of a structure definition.
- Comparing two structure variables with == ; compare member by member instead.
- Using & with a character array in scanf (write s[i].name, not &s[i].name).
- Reading a string longer than the array; use a width limit such as %19s for name[20].
- Using the dot operator on a pointer (write p->roll or (*p).roll).

3. Stacks

3.1 Definition

A **stack** is a linear data structure in which insertion and deletion are done only at one end, called the **top**. The element inserted last is removed first, so a stack follows **LIFO: Last In, First Out**. A pile of plates is the usual example: you add a plate on top and take a plate from the top.

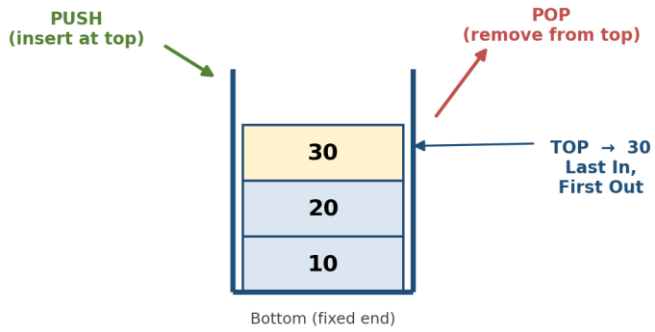


Fig. 9: Stack: LIFO principle (push and pop at the top)

3.2 Stack operations

Operation	Purpose	Condition / note
push(x)	Insert x at the top	Overflow if the stack is full (top == MAX-1)
pop()	Remove and return the top element	Underflow if the stack is empty (top == -1)
peek() / top	Read the top element without removing it	Stack must not be empty
isEmpty()	Check whether the stack has no elements	True when top == -1
isFull()	Check whether no more elements can be added	True when top == MAX-1

3.3 Array representation of a stack

A stack can be stored in an array stack[MAX] with an integer top that holds the index of the topmost element. Initially top = -1, which means the stack is empty.

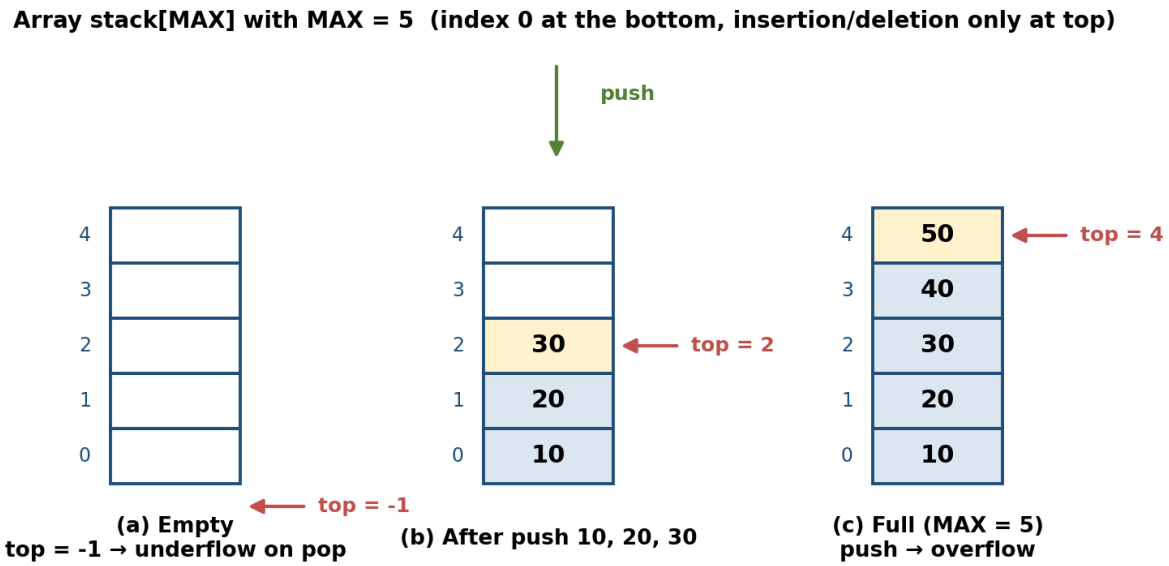


Fig. 10: Array representation of a stack: empty, partly filled, full

push(x):

1. If top == MAX-1, report Stack Overflow and stop.
2. Increment top.
3. Store x in stack[top].

pop():

1. If top == -1, report Stack Underflow and stop.
2. Read x = stack[top].
3. Decrement top and return x.

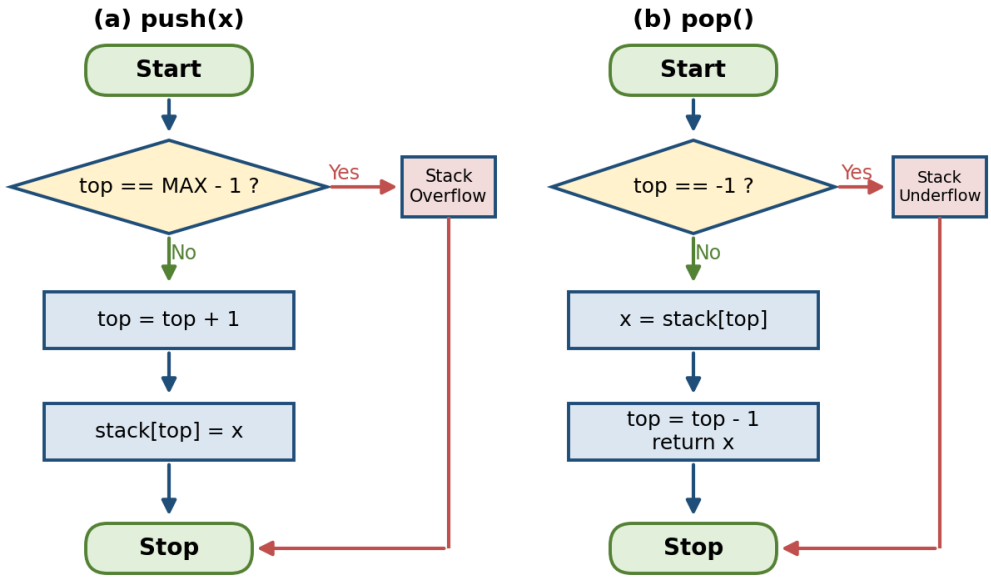


Fig. 11: Flowcharts for push(x) and pop()

Complete C program:

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int x)
{
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = x;
}

int pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;                  /* -1 signals an error here */
    }
    return stack[top--];
}

int peek()
{
    if (top == -1)
    {
        printf("Stack is empty\n");
        return -1;
    }
    return stack[top];
}
```

```
void display()
{
    int i;
    if (top == -1)
        printf("Stack is empty\n");
    else
        for (i = top; i >= 0; i--)
            printf("%d\n", stack[i]);
}

int main()
{
    push(10);
    push(20);
    push(30);
    printf("Popped: %d\n", pop());
    push(40);
    printf("Top: %d\n", peek());
    display();
    return 0;
}
```

Output of the program:

```
Popped: 30
Top: 40
40
20
10
```

3.4 Trace of stack operations

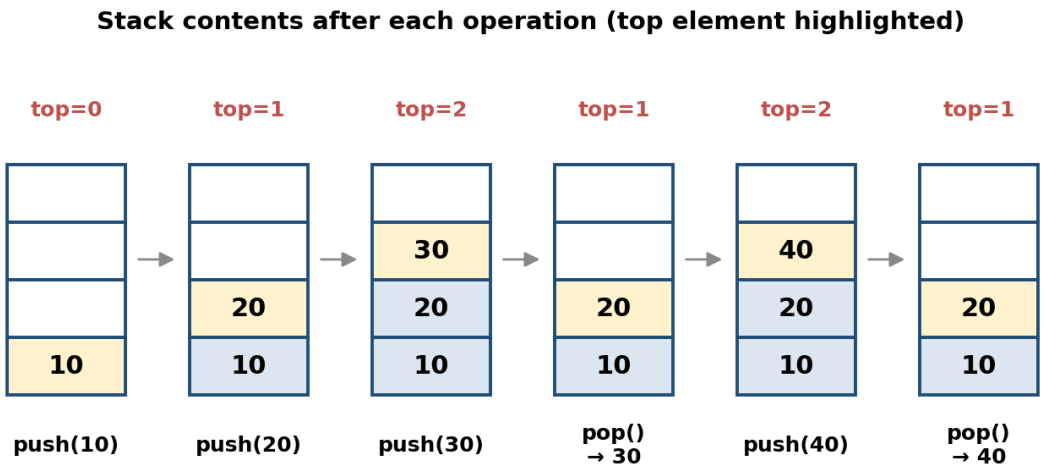


Fig. 12: Trace of stack operations (matches the table below)

Operation	top	Stack (bottom → top)	Result
Start	-1	empty	–
push(10)	0	10	–
push(20)	1	10, 20	–
push(30)	2	10, 20, 30	–
pop()	1	10, 20	returns 30
push(40)	2	10, 20, 40	–
peek()	2	10, 20, 40	returns 40
pop()	1	10, 20	returns 40

3.5 Complexity and limitations

- push, pop, peek, isEmpty and isFull each take O(1) time because only the top index changes.
- An array stack has a fixed capacity: pushing into a full stack causes overflow even if memory is available elsewhere.
- If MAX is chosen too large, memory is wasted; a linked-list stack removes this limit (studied later).

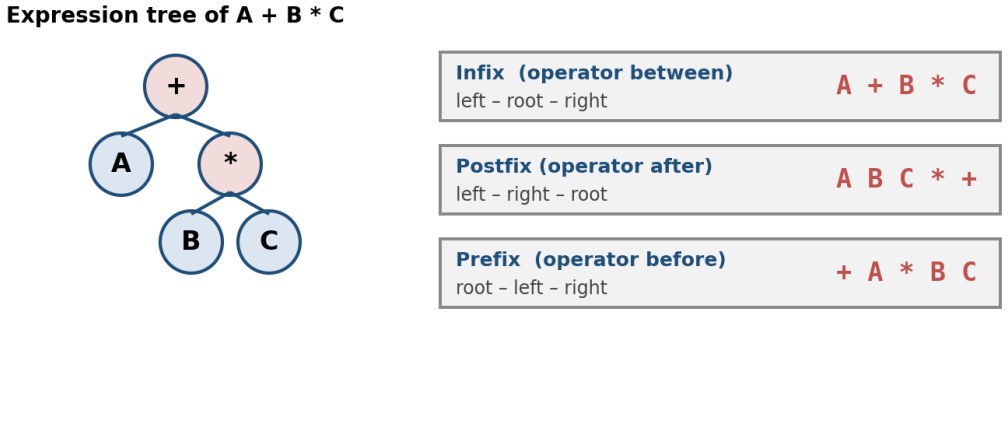
3.6 Common mistakes

- Pushing without checking for overflow, or popping without checking for underflow.
- Initializing top to 0 while still using top as the index of the last element (the empty stack must be top = -1).
- Returning a value such as -1 for underflow when -1 can also be a valid stack element.

4. Types of expressions: infix, postfix and prefix

4.1 Three notations

Notation	Operator position	Example	Needs parentheses / precedence rules?
Infix	Between the operands	A + B * C	Yes
Postfix (suffix)	After the operands	ABC*+	No; order of operators decides evaluation
Prefix (Polish)	Before the operands	+A*BC	No; order of operators decides evaluation



** is evaluated before + because it sits lower in the tree (higher precedence)*

Fig. 13: Expression tree and the three notations

4.2 Precedence and associativity

Operator	Precedence	Associativity
^ (exponent)	Highest	Right to left
* / %	Middle	Left to right
+ -	Lowest	Left to right
( )	Overrides all of the above	Innermost first

4.3 Manual conversion by full parenthesization

Before using the stack algorithms, expressions can be converted by hand. Fully parenthesize the infix expression according to precedence, then move each operator: for **postfix**, to just after its right operand (at its closing bracket); for **prefix**, to just before its left operand (at its opening bracket). Finally remove the brackets.

Infix	Fully parenthesized	Postfix	Prefix
A + B * C	(A + (B * C))	ABC*+	+A*BC
(A + B) * (C - D)	((A + B) * (C - D))	AB+CD-*	*+AB-CD
A * B + C / D	((A * B) + (C / D))	AB*CD/+	+*AB/CD
(A + B) * C - D / E	(((A + B) * C) - (D / E))	AB+C*DE/-	-*+ABC/DE

4.4 Why postfix and prefix are useful

- No parentheses and no precedence rules are needed once the expression is in postfix or prefix form.
- A postfix expression can be evaluated in a single left-to-right scan using a stack; compilers and calculators use this idea.
- The stack-based conversion algorithms are in Sections 8 and 9, and postfix evaluation is in Section 10.

5. Case study: real-world applications

5.1 Where these data structures are used

Data structure	Real-world application
Array	Marks of a class, pixels of an image, price list, lookup tables
Structure / array of structures	Student, employee, patient or inventory records; library catalogue entries
Stack	Undo/redo in editors, browser back button, function-call stack, checking balanced brackets, expression evaluation in compilers and calculators
Trees and graphs (later modules)	File systems, organization charts, road and network maps

5.2 Case study: Undo in a text editor

Every edit is pushed on an undo stack. Pressing Undo pops the most recent edit and reverses it, which is exactly LIFO behaviour: the last change made is the first one undone. (Redo works with a second stack that receives the popped edits.)

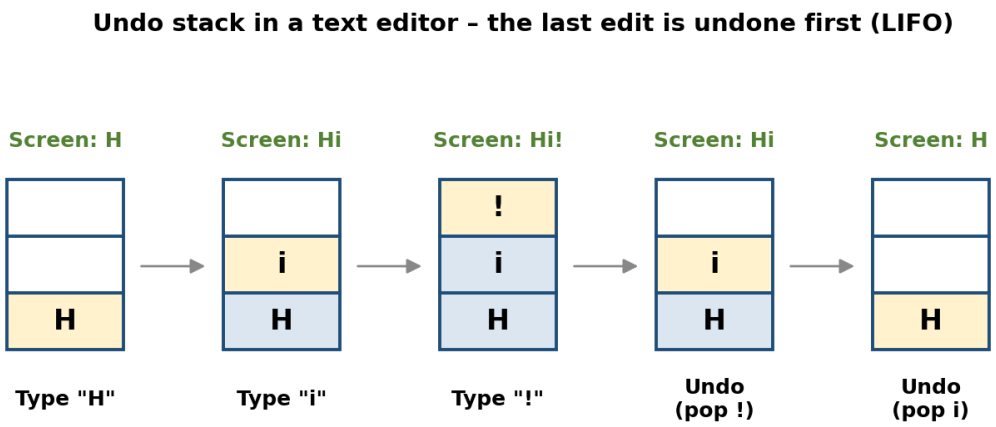


Fig. 14: Undo stack in a text editor

Action	Undo stack (bottom → top)	Text on screen
Type "H"	H	H
Type "i"	H, i	Hi
Type "!"	H, i, !	Hi!
Undo	H, i	Hi
Undo	H	H

5.3 Case study: Checking balanced parentheses

Compilers must check that every opening bracket has a matching closing bracket. Scan the expression from left to right: push every (; on) pop one element (if the stack is empty at that moment, the expression is unbalanced). At the end the expression is balanced only if the stack is empty.

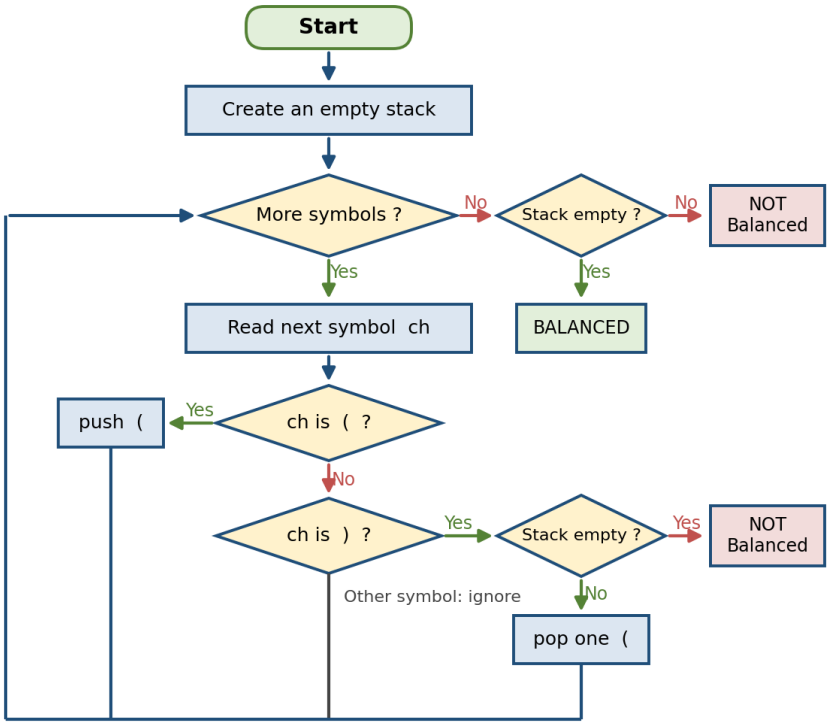


Fig. 15: Flowchart: checking balanced parentheses with a stack

Symbol	Action	Stack
(	Push	(
A + B	Ignore operands and operators	(
)	Pop	empty
* C	Ignore	empty
(	Push	(
End of input	Stack not empty	(→ not balanced

The expression (A+B)*(C is therefore reported as unbalanced because one (was never closed.

5.4 Case study: Student record system

A small college application stores each student as a structure and all students in an array of structures. The five basic operations map directly to features of the application:

- **Traversing**: print the class list or compute the class average.
- **Inserting**: admit a new student at the end of the array.
- **Deleting**: remove a student who has left; later records shift left.
- **Searching**: find a student by roll number.
- **Sorting**: produce a rank list ordered by marks.

6. Quick revision and practice questions

6.1 Key points

- A data structure organizes data for efficient operations; ADTs describe behaviour, not implementation.
- Linear structures: array, stack, queue, linked list. Non-linear: tree, graph.

- Basic operations: traversing, inserting, deleting, searching, sorting. Array insertion and deletion take O(n) in the worst case.
- A structure groups members of different types; use . with variables and -> with pointers.
- A stack is LIFO. In the array form, top = -1 means empty and top = MAX-1 means full; all operations are O(1).
- Infix needs precedence and brackets; postfix and prefix do not.

6.2 Practice questions

Short answer (2 marks)

- Define a data structure. Differentiate between primitive and non-primitive data structures.
- List the five basic data structure operations.
- What is the difference between a structure and a union?
- What are stack overflow and stack underflow?

Long answer (5 to 10 marks)

- Classify data structures with examples and explain any four operations on an array.
- Explain structures in C. Write a program to read and display details of n students using an array of structures.
- Explain the array representation of a stack and write push, pop and display functions with overflow and underflow checks.
- Explain infix, postfix and prefix expressions with examples and show how precedence affects conversion.

Problems with answers

Problem	Answer
Insert 25 at index 2 of 10, 20, 30, 40	10, 20, 25, 30, 40
Stack (MAX = 3): push 1, push 2, push 3, push 4	4th push gives Stack Overflow; stack stays 1, 2, 3
Stack is empty and pop() is called	Stack Underflow
Postfix of (A + B) * C - D / E	AB+C*DE/-
Prefix of (A + B) * C - D / E	-*+ABC/DE

MODULE II – STACK APPLICATIONS (INFIX TO POSTFIX / PREFIX, POSTFIX EVALUATION, RECURSION)

7. Stack applications and expression forms

7.1 Basic terms

An expression combines operands (values or variables) and operators such as +, -, *, /, % and ^. A stack follows LIFO: Last In, First Out.

Form	Operator position	Example	Typical use
Infix	Between operands	A + B	Human-readable mathematics/programming
Postfix / Suffix	After operands	AB+	Stack-based evaluation
Prefix	Before operands	+AB	Expression processing/parsing

Typical precedence: ^ highest; then *, /, %; then +, -. Parentheses override normal precedence. Exponentiation is usually right-associative; the other arithmetic operators are usually left-associative.

7.2 Conversion examples

Infix	Postfix	Prefix
A + B	AB+	+AB
A + B * C	ABC*+	+A*BC
(A + B) * C	AB+C*	*+ABC
A + B * (C - D)	ABCD-*+	+A*B-CD

8. Infix to postfix conversion

8.1 Idea

Scan the expression left to right. Output operands immediately. Keep operators on a stack until their precedence says they can be output.

8.2 Algorithm

1. Initialize an empty operator stack and an empty output string.
2. Read the next symbol.
3. If it is an operand, append it to output.
4. If it is '(', push it.
5. If it is ')', pop to output until '(' is found, then discard the '(' .
6. If it is an operator, pop higher-precedence operators from the stack; also pop equal-precedence operators when the current operator is left-associative. Then push the current operator.
7. Repeat for all symbols.
8. Pop remaining operators to output. Unmatched parentheses indicate an invalid expression.

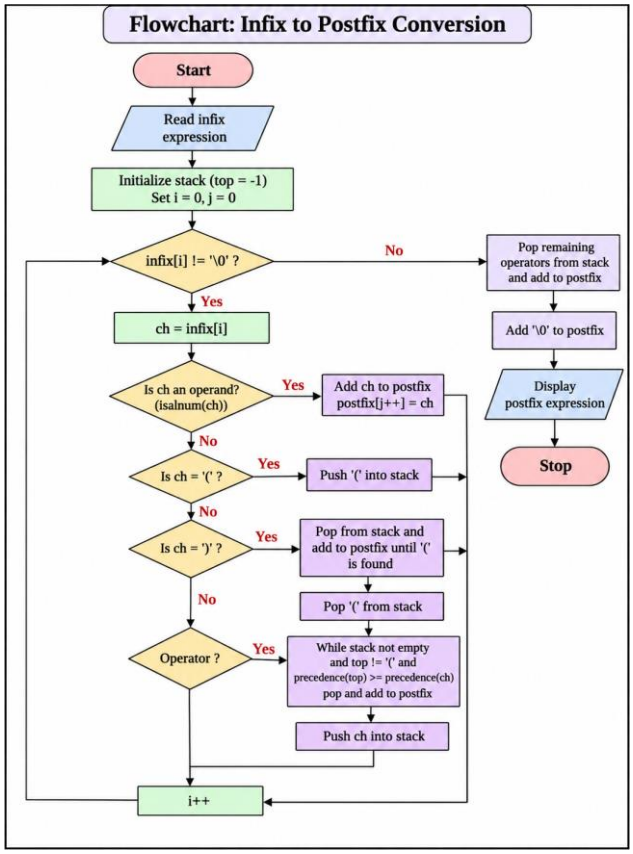
8.3 Worked example: A + B * (C - D)

Symbol	Action	Stack (bottom → top)	Output
A	Output operand	—	A

+	Push	+	A
B	Output operand	+	AB
*	Push	+ *	AB
(	Push	+ * (	AB
C	Output operand	+ * (	ABC
-	Push	+ * (-	ABC
D	Output operand	+ * (-	ABCD
)	Output -; discard (	+ *	ABCD-
End	Output * then +	—	ABCD-*+

Result: ABCD-*+. A stack is useful because operators must wait until their operands/subexpressions are ready and precedence is respected.

8.4 Flowchart outline



8.5 Common mistakes

- Outputting parentheses in postfix.
- Forgetting to pop remaining operators at the end.
- Ignoring precedence or associativity.
- Using a simple character scanner for expressions with spaces, unary minus or multi-digit numbers without adding tokenization.

9. Infix to prefix conversion

9.1 Standard classroom method

1. Reverse the infix expression.

2. Swap every '(' with ')' and every ')' with '('.
3. Convert the modified expression to postfix, carefully adjusting associativity rules for the reversed expression.
4. Reverse the resulting postfix expression to get prefix.
5. Check the final grouping of operands and operators.

9.2 Example

Infix: (A + B) * C

Reverse and swap parentheses: C * (B + A)

Postfix of modified expression: CBA+*

Reverse it: *+ABC

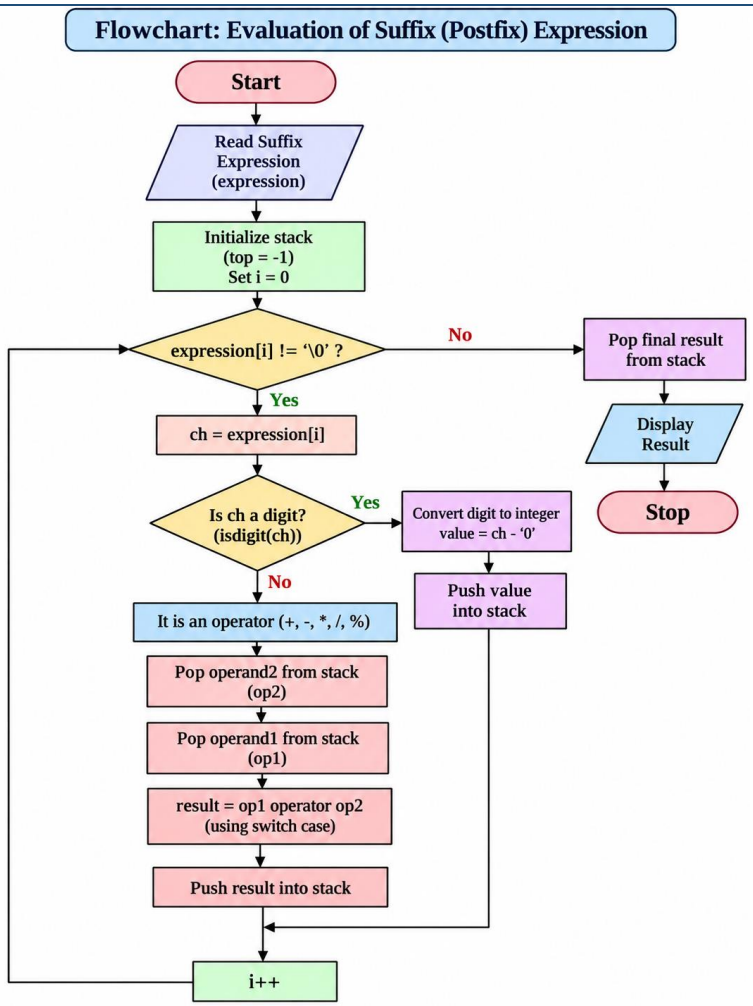
Prefix result: *+ABC

Caution: expressions containing ^ or equal-precedence operators require correct associativity handling; verify the result rather than applying reversal mechanically.

10. Postfix / suffix expression evaluation

10.1 Algorithm

1. Create an empty stack of values.
2. Scan tokens from left to right.
3. If a token is an operand, convert it to a value and push it.
4. If a token is an operator, pop operand2 first, then operand1.
5. Calculate operand1 operator operand2.
6. Push the result.
7. After scanning, exactly one value should remain; this is the answer. Check for insufficient operands, extra values and division by zero.



10.2 Worked example: 23*54*+

Token	Action	Stack (bottom → top)
2	Push 2	2
3	Push 3	2, 3
*	2 × 3 = 6	6
5	Push 5	6, 5
4	Push 4	6, 5, 4
*	5 × 4 = 20	6, 20
+	6 + 20 = 26	26

Final result = 26.

10.3 Operand order

For 82-, pop 2 first as operand2 and 8 second as operand1. Calculate 8 − 2 = 6. This order matters for subtraction and division.

10.4 Implementation note

A character-by-character C program supports single-digit operands such as 23*. To handle multi-digit numbers or identifiers, tokenize the expression, for example using spaces between tokens. Check stack underflow and division by zero.

11. Recursion

11.1 Definition and parts

Recursion occurs when a function calls itself to solve a smaller instance of the same problem. Every correct recursive solution needs a base case (stopping condition) and a recursive case (a call that moves toward the base case).

11.2 Factorial example

```
int factorial(int n)
{
          if (n <= 1)
                return 1;   // base case
return n * factorial(n - 1);
}
```

factorial(4) = 4 × factorial(3) = 4 × 3 × factorial(2) = 4 × 3 × 2 × factorial(1) = 24.

11.3 Sum from 1 to n

```
int sum(int n)\n{\n    if (n <= 0) return 0;\n    return n + sum(n - 1);\n}
```

sum(4) = 4 + 3 + 2 + 1 + 0 = 10.

11.4 Recursion and the call stack

Each function call stores information such as parameters, local variables and return location in a call-stack frame. Calls return in LIFO order.

11.5 Uses and limitations

- Useful for tree traversals, factorial, divide-and-conquer and backtracking.
- Can make a naturally recursive problem easier to express.
- Function calls consume time and stack memory.
- Deep or infinite recursion may cause stack overflow; ensure each call progresses toward a base case.